

MethodMesh v1.05

Module Review and Refresh Manual

Migration instructions for reviewing and updating existing MethodMesh modules

Standard	MethodMesh Master Book v1.05
Purpose	Repeatable migration and QA of existing modules
Primary deliverable	One completed module folder only

Core principle: one canonical capability contract, projected consistently across dashboard, direct native use, presets, protocols, schedules/widgets where applicable, and ODK/XLSForm.

1. Purpose

You are reviewing an existing MethodMesh module and updating it to conform to MethodMesh Master Book v1.05.

This is a migration and quality-review task, not a greenfield rewrite.

Your job is to:

- understand what the existing module already does;
- preserve its useful functionality and established contracts;
- identify where it violates or predates v1.05;
- update the module to the current MethodMesh architecture and UX;
- verify all capability surfaces remain available;
- return only the completed module folder.

The supplied MethodMesh Master Book v1.05 is authoritative. Do not invent a different architecture because you prefer it.

2. The central architectural rule

Every MethodMesh capability has one canonical capability contract and multiple interaction surfaces.

The capability itself is the product primitive. The following are projections or invocation routes over that same capability:

- native direct use;
- module dashboard;
- presets;
- protocols;
- schedules where applicable;
- Android widgets where applicable;
- ODK/XLSForm;
- external invocation where supported.

Do not implement separate versions of the capability for different surfaces. Do not make functionality private to the dashboard. Do not make a capability available through native UI but unavailable to presets, protocols or ODK.

Required invariant

If MethodMesh can perform an operation, the underlying capability must remain independently addressable through the canonical capability contract.

If MethodMesh can produce an output, that output must remain available through the contract and therefore be returnable to ODK where technically representable.

This applies even to obscure outputs that normal users will rarely request. "Beef first" is a presentation rule, not permission to delete outputs from the capability contract.

3. Your input

You will normally receive:

- the existing module folder;
- MethodMesh Master Book v1.05;
- optionally build errors, screenshots or known issues.

Start by examining the existing module. Do not immediately rewrite it.

Determine:

- module name;
- capabilities/method IDs;
- existing settings;
- runtime inputs;
- outputs;
- dashboard implementation;
- native screens;
- preset behaviour;
- protocol exposure;
- ODK/XLSForm support;
- widget/schedule behaviour if present;
- persistence/state requirements;
- dependencies;
- documentation;
- tests;
- known TODOs.

4. Preserve contracts unless there is a compelling reason not to

Existing method IDs and established output field names may already be used by saved presets, protocols, XLSForms, external integrations, tests and user workflows.

Do not casually rename

- method IDs;
- setting keys;
- input keys;
- output keys;
- namespace behaviour.

If an existing name is ugly but functional, preserving compatibility is normally more important than cosmetic purity. If a contract genuinely must change, document the old field, new field, reason and compatibility implications. Prefer backwards-compatible aliases where practical.

5. First review: capability inventory

Before modifying code, make a private inventory of every capability in the module. For each capability identify:

Area	Determine
Method ID	Canonical identifier
Purpose	What useful task it performs
Inputs	Runtime data supplied by user/external caller
Settings	Reusable configuration
Operational controls	Things such as reconnect/select device
Main result	Primary "beef"
Secondary outputs	Other useful calculated/returned values

Audit outputs	Time, hashes, IDs, provenance, diagnostics
Media outputs	Images, maps, PDFs, files, audio etc.
Native UI	Current interaction
Preset exposure	Yes/no
Protocol exposure	Yes/no
ODK exposure	Yes/no
Dashboard presence	Yes/no
Persistent state	Whether required
Offline behaviour	Expected behaviour

Do not allow the dashboard to obscure the fact that a module may contain several independent capabilities.

6. Review the native UX

MethodMesh v1.05 moves away from the generic pattern:

settings form -> Go -> generic result page

That should no longer be the normal production experience. The preferred lifecycle is:

configure / interact -> see live current result -> Commit -> share/save/finish

Capability-specific interface

Production capabilities should have a UI appropriate to what they do.

- music tools should look and behave like music tools;
- maps should behave like maps;
- counters like counters;
- timers like timers;
- scoreboards like scoreboards;
- image tools should centre the image;
- measurement tools should centre the measurement;
- text processors should centre editable/output text;
- sensor tools should show live sensor values clearly.

MethodSetting metadata remains the canonical configuration contract. It does not mean the production UI must resemble an automatically generated settings form.

Generic forms are acceptable for development, fallback, obscure technical settings, and preset configuration where appropriate. They should not substitute for a polished production interaction when a better UI is obvious.

7. Working results should be live

Where output can sensibly be calculated while the user works, show it on the same screen.

- changing a musical key immediately changes displayed notes/chords;
- changing two polyrhythm values immediately updates the ratio and pattern;
- changing dimensions immediately updates derived measurements;

- moving on a map updates coordinates/Plus Code;
- adjusting score changes totals;
- changing text updates transformed output;
- sensor readings refresh in place.

Do not require the user to press Calculate and travel to another generic screen merely to see deterministic calculations that can be updated immediately.

A deliberate action button is still appropriate when an actual operation must occur, for example:

- Scan;
- Capture;
- Record;
- Measure;
- Query;
- Download;
- Randomise;
- Start;
- Stop;
- Connect.

The resulting output should normally still appear within the working screen.

8. Working state versus committed result

Working state

The user is still changing inputs or interacting with the capability. Results may change continuously.

Committed state

The user says Commit. Commit means:

- freeze the canonical result for this execution;
- establish the output payload;
- make result actions available;
- allow the result to be returned to its caller.

Commit does not normally mean "navigate to an ugly generic result page".

After Commit, expose appropriate actions such as Share, Save, Copy, Include full JSON/audit, Done/Home, and Edit/Retry where appropriate.

If the user edits the working state after committing, do not silently mutate a previously committed result without an explicit recommit.

9. Tap-to-copy is universal for useful displayed values

Any calculated or returned scalar/text value displayed in normal MethodMesh UI should normally be directly tappable to copy.

- Plus Code;
- temperature;
- humidity;
- dew point;
- distance;
- bearing;
- score;
- ratio;

- calculated value;
- selected note/chord;
- transformed text;
- hash;
- coordinate;
- identifier where useful.

Clipboard content should be the useful value, not the UI label.

Good: 8FVC9G8F+6W
Bad: Plus Code: 8FVC9G8F+6W

Good: 3
Bad: music_polyrhythm_ratio_a: 3

Use sensible user-facing formatting. Machine field names belong in technical details, not the primary UI.

10. Dashboard requirements

Every normal module should have an appropriate dashboard presence. The dashboard should act as a useful module control centre where sensible.

It may show:

- current values;
- useful summaries;
- live status;
- quick controls;
- recent useful output;
- active schedules;
- launch controls;
- maps/images;
- preset shortcuts.

The dashboard is only one way to interact with a module.

Never make the dashboard the only place where a capability exists.

Dashboard aggregation must not remove capability granularity

If a module dashboard combines stopwatch, countdown timer and interval timer, those underlying capabilities must still independently exist for preset creation, protocol composition and ODK invocation.

Likewise, a sophisticated module control centre must not turn its component capabilities into private implementation details.

11. Preset requirements

Every capability that makes sense as a reusable MethodMesh operation must remain individually available for preset creation.

Preset configuration distinguishes fixed configuration, runtime inputs and operational controls.

When a preset runs:

- fixed values should normally be hidden;
- runtime inputs should be requested;
- if all required values are fixed, run directly where appropriate;
- the same capability UI/result lifecycle should be used rather than an inferior second implementation.

Do not expose only the module dashboard as a preset. Individual capabilities remain composable.

12. Protocol requirements

Each capability must remain available as an independent protocol step where meaningful. Protocol execution must use the same canonical method contract.

The protocol runner supplies previous/next rails, chaining, pipe resolution, retry/cancel behaviour and combined final results. The capability itself supplies its actual operation.

Do not duplicate the capability inside protocol code. Do not require protocols to launch the module dashboard unless the dashboard itself is genuinely the intended capability.

13. ODK/XLSForm parity is mandatory

ODK is a first-class invocation route. Anything that MethodMesh can do through a declared capability should be callable from ODK. Anything that MethodMesh can return through that capability should be returnable to ODK.

This includes:

- main result;
- secondary values;
- obscure calculated outputs;
- metadata;
- audit outputs;
- hashes;
- diagnostics where declared;
- structured JSON;
- media/file outputs.

Example XLSForms may demonstrate only the most useful subset. They are examples, not allow-lists.

13.1 Non-interactive ODK route

ODK -> MethodMesh capability -> execute -> return result to ODK

If ODK provides everything needed, do not force the user through native setup screens. Do not save extra copies merely because ODK invoked the method.

13.2 Interactive ODK route

Some operations necessarily require MethodMesh interaction, such as selecting an area on a map, framing a photograph, redacting an image, scanning something, or interacting with a visual instrument.

ODK -> launch appropriate MethodMesh capability UI -> user interacts -> Commit -> result returns directly to ODK

The user should not then encounter MethodMesh normal Share, Save or Home flow unless explicitly relevant. ODK owns the surrounding submission.

13.3 Transport rules

Preserve:

- grouped intent invocation;
- input_* parameters;
- canonical returned keys;
- namespace projection;
- methodmesh_full_json;
- ClipData;
- URI read grants;
- binary artefact attachment semantics.

Do not solve capability-specific ODK issues by teaching shared UI bespoke module behaviour.

14. Widgets

Widget-launched actions use the same canonical capabilities. After completion, return gracefully to the Android desktop.

Do not leave the user stranded inside MethodMesh navigation. A widget must not introduce a third implementation of a capability. Launch origin should be tracked so completion routing behaves correctly.

15. Home / Done / Commit routing

Launch origin	Completion destination
Normal app/direct capability	Done/Home -> MethodMesh dashboard.
Preset launched in app	Done -> MethodMesh dashboard.
Widget	Done -> Android desktop.
Protocol	Completion -> protocol runner continues.
Schedule	Completion -> schedule machinery handles closeout.
ODK/external roundtrip	Commit -> return canonical payload to caller.

Do not blindly use the same navigation destination everywhere.

16. Remove generic framework clutter from normal capability UI

Review for artefacts such as:

- Step 1 of 1;
- raw method IDs;
- raw output keys;
- huge generic Result boxes;
- redundant Home button at the top;
- disabled Done buttons with unclear meaning;
- giant stacks of generic Share/Copy/Save controls;
- verbose technical metadata;
- debug-oriented labels.

These may be useful during development but should not dominate production UX. Move technical material behind Technical details or equivalent progressive disclosure.

17. Media and maps

For visual outputs:

- image/map/document should be central to the UI;
- provide compact share/save controls;
- do not wrap them in unnecessary generic result text.

Maps should support:

- smooth pan;
- smooth zoom;
- appropriate gestures;
- full-screen viewing;
- preservation of state on return.

Map capabilities must still expose their canonical values independently of their visual presentation.

18. Persistent state

Ask whether the operation may realistically last long enough to survive rotation, activity recreation, app backgrounding, accidental navigation, or process recreation where reasonable.

Persistent state is particularly important for:

- timers;
- scoreboards;
- games;
- counters;
- long measurements;
- ongoing recording;
- navigation;
- campaign/session tools;
- multi-stage work.

Do not store things permanently merely because persistence is useful during an active execution. Distinguish transient UI state, active-session persistent state, committed output and deliberately saved records.

19. Module-local ownership

The module should own its own:

- method descriptors;
- settings;
- screens;
- dashboard components;
- output definitions;
- helpers;
- repository/state code where required;
- documentation;
- example XLSForms;
- tests where appropriate.

The shared MethodMesh app must not acquire capability-specific knowledge.

Ask: Is this change useful generically to many unrelated capabilities? If yes, it may belong in shared infrastructure. If no, keep it inside the module.

20. Handoff folder structure

Return one module folder only. The archive/deliverable root must be:

```
<module_name>/
```

Not:

```
MethodMesh/  
  app/  
    src/  
      main/  
        java/  
        ...
```

Not:

```
app/  
src/  
...
```

Not:

```
modules/  
  <module_name>/
```

The returned object itself is <module_name>/. A typical module looks like:

```
<module_name>/  
|-- docs/  
|   |-- README_<ModuleOrCapability>.md  
|   |-- example_odk_<capability>.xlsx  
|   |-- example_odk_<second_capability>.xlsx  
|   |-- VALIDATION.md  
|   |-- ROADMAP_NOTE.md  
|   |-- ATTRIBUTION.md  
|   `-- THIRD_PARTY_NOTICES.md  
|  
|-- <ModuleName>Module.kt  
|-- <CapabilityName>Method.kt  
|-- <CapabilityName>CapabilityScreen.kt  
|-- <SecondCapability>Method.kt  
|-- <SecondCapability>CapabilityScreen.kt  
|  
|-- <ModuleName>Dashboard.kt  
|-- <ModuleName>Repository.kt  
|-- <ModuleName>State.kt  
`-- tests or module-owned helpers where appropriate
```

Only include files actually required by the module. Do not create empty ceremonial files.

21. Do not return repository clutter

Never include:

- whole MethodMesh repository tree;
- app/;
- .gradle/;
- build/;
- .idea/;
- .git/;
- local.properties;
- generated APKs;
- unrelated capabilities;
- unrelated shared UI;
- unrelated Gradle files;
- random screenshots;
- OS metadata;
- temporary files.

If returning a ZIP, opening the ZIP should reveal exactly one top-level <module_name>/ directory.

Do not use a redundant wrapper such as:

```
music_module.zip  
`-- music_module_delivery/  
    |-- music_module/
```

22. Shared-framework changes

Occasionally a legitimate v1.05 migration may reveal a missing generic framework feature. Examples might include generic Commit lifecycle support, launch-origin handling, tap-to-copy result metadata, ODK projection or dashboard action declarations.

Do not silently include a rewritten framework inside the returned module. Instead:

- keep the module self-contained as far as possible;
- document the framework dependency clearly;
- state exactly what generic framework change is required;
- explain why it cannot reasonably be implemented module-locally.

The actual handoff remains the module folder.

23. Documentation update

Update the module README to describe current reality. At minimum document:

- purpose;
- method IDs;
- capability list;
- status/lane;
- dashboard behaviour;
- direct native behaviour;
- preset behaviour;
- protocol behaviour;
- ODK behaviour;
- widget/schedule behaviour if relevant;
- settings;
- runtime inputs;
- outputs;
- primary result;
- secondary outputs;
- audit outputs;
- full JSON;
- media;
- offline/online behaviour;
- permissions;
- persistence;
- dependencies;
- attribution/licensing;
- validation status.

Do not document aspirational behaviour as already implemented.

24. Example XLSForms

Where ODK use is plausible, include working example XLSForm(s). For multi-capability modules, examples must adequately demonstrate the module capabilities.

Do not assume one dashboard-oriented XLSForm substitutes for demonstrating individually callable capabilities.

Use:

- grouped intent calls;

- correct input fields;
- correct return fields;
- namespace handling;
- media fields where relevant;
- methodmesh_full_json where appropriate.

The example may stay concise. The actual contract may expose many more outputs than the demonstration form uses.

25. Testing expectations

At minimum review/test the contract boundaries most affected by the migration.

Contract parity

For each method:

- discoverable as native capability;
- available to presets;
- available to protocols;
- ODK callable;
- declared outputs are projectable to ODK.

Where feasible, this should be programmatically enumerated rather than manually asserted.

UX/state

- live result updates;
- Commit freezes result;
- tap-to-copy uses correct value;
- rotation preserves working state;
- committed state survives rotation;
- Retry/Edit semantics are clear;
- Done routing respects launch origin.

ODK

- input projection;
- output projection;
- namespaces;
- JSON;
- media URI handling;
- read grants;
- no accidental extra persistence.

Build

```
./gradlew :app:testDebugUnitTest
./gradlew :app:assembleDebug
```

A module that does not build must not be represented as production-ready.

26. Review severity

Pass A - Contract correctness

Look for architecture failures:

- capability only exists on dashboard;
- missing preset exposure;

- missing protocol exposure;
- ODK cannot invoke capability;
- ODK cannot return declared outputs;
- duplicated implementations;
- broken method IDs;
- output contract inconsistencies.

Fix these first.

Pass B - Interaction quality

Look for:

- settings form -> generic result page;
- deterministic outputs requiring unnecessary Calculate;
- results not visible while working;
- generic machine labels;
- unclear Commit semantics;
- bad completion routing;
- values not tappable to copy;
- clunky visual design.

Bring these to v1.05.

Pass C - Field robustness

Look for:

- rotation loss;
- lost active session;
- accidental auto-save;
- unclear offline failure;
- media URI problems;
- navigation traps;
- widget completion problems;
- ODK roundtrip problems;
- inaccessible controls;
- excessive verbosity.

27. Do not over-redesign

A v1.05 refresh does not mean changing the purpose of the module.

Do not:

- invent unrelated capabilities;
- remove niche capabilities because you personally find them obscure;
- simplify away outputs;
- replace local/offline functionality with cloud services;
- introduce central special-casing;
- rename stable contracts gratuitously;
- redesign the whole app.

Improvements that clearly complete the existing concept are reasonable. Major new scope belongs in a roadmap note unless explicitly requested.

28. Definition of done

A module is ready to return only when the answer to each applicable statement is yes.

Architecture

- The module implements one canonical contract per capability.
- No capability is dashboard-only.
- Individual capabilities remain available to presets.
- Individual capabilities remain available to protocols.
- ODK can invoke each capability.
- ODK can request all declared returnable outputs.
- Shared UI contains no new module-specific special cases.

Native UX

- The module has an appropriate polished dashboard presence.
- Each capability has an appropriate production interface.
- Live/current results appear on the working screen where sensible.
- Commit clearly finalises the result.
- Result actions appear after Commit rather than requiring a generic result page.
- Useful displayed values support tap-to-copy.
- Technical details do not dominate the primary interface.

Completion

- App-native completion returns to MethodMesh appropriately.
- Protocol completion returns to the protocol runner.
- ODK completion returns to ODK.
- Widget completion returns to Android desktop.
- Schedule completion behaves appropriately.

Robustness

- Relevant state survives rotation.
- Long-running state persists where necessary.
- No unwanted automatic storage occurs.
- Offline behaviour is sensible and documented.

Documentation

- Module README reflects actual behaviour.
- Example XLSForm(s) are present where appropriate.
- Output contract is documented.
- Permissions/dependencies/attribution are documented.

Delivery

- Only one <module_name>/ folder is returned.
- No fake whole-app tree is returned.
- No build/IDE/repository clutter is included.

29. Required final self-review

- Can I perform every capability directly?
- Can I make a preset from every appropriate capability?
- Can I use every appropriate capability as a protocol step?

- Can ODK invoke every capability?
- Can ODK receive every output that the capability declares?
- Is the dashboard a polished control centre without becoming the capability implementation itself?
- Does the normal native UI feel like a purpose-built tool rather than a generated settings form?
- Do current results appear while I am working?
- Does Commit mean something clear?
- Can I tap useful values to copy them?
- Does completion return to the correct place for app, protocol, ODK and widget launches?
- Have I preserved stable method IDs and contracts?
- Am I returning exactly one module folder?

If any answer is no, the migration is not complete.

30. Required response format

Do not spend the final response giving a long explanation of what you changed. Return the updated module folder.

A concise note may identify important compatibility decisions, a remaining framework dependency, or anything that could not be tested. But the principal deliverable is the folder itself.

Do not return a tree for the whole application. Do not return loose files. Return the module folder.

31. Short prompt to place above this manual

For running repeated migrations, use the following prompt with the module folder and this manual:

Review and update the attached MethodMesh module to MethodMesh Master Book v1.05. Follow the attached Module Review and Refresh Manual exactly. This is a migration of the existing module, not a greenfield redesign. Preserve established method IDs and contracts unless genuinely broken. Pay particular attention to canonical capability parity across dashboard, direct native use, presets, protocols and ODK; the v1.05 live-working-result -> Commit lifecycle; tap-to-copy; launch-origin-aware completion; polished capability-specific UI; state preservation; and ODK roundtrip completeness. Build/test where possible. Return only the finished module folder, never a whole-app tree.

32. Recommended migration scorecard

For a large migration programme, maintain one row per module so that "reviewed" means more than simply "a chat worked on it".

Module	Capabilities	Contract parity	UI migrated	ODK verified	Persistence verified	Tests/build	Framework dependency